二、Agent

1.Agent概念

1.1 前情回顾

在前面的章节中，我们已经系统介绍了Tools的概念及其核心价值。Tools作为大型语言模型（LLM）能力边界的重要拓展机制，使得模型能够突破单纯依赖预训练知识的局限，通过调用外部函数、API或资源来完成更复杂的任务，例如实时查询数据、执行代码计算或与外部系统交互。这不仅显著提升了LLM的实用性，还为其注入了动态处理现实世界问题的能力。

1.1.1 存在的问题

然而，单纯引入Tools并不足以让LLM达到真正智能的水平。

我们可以形象地将Tools比喻为给LLM“添加了手脚”——它获得了执行具体动作的能力，但这种能力仍处于“遥控机械臂”的阶段。

这种模式下，模型难以实现“如臂使指”的自然流畅性：每一次工具调用都需要明确的外部提示引导，缺乏自主性和灵活性。当任务较为简单时，这一局限性可能并不明显；但在实际应用场景中，尤其涉及多种工具的交互使用时，问题就会凸显出来。

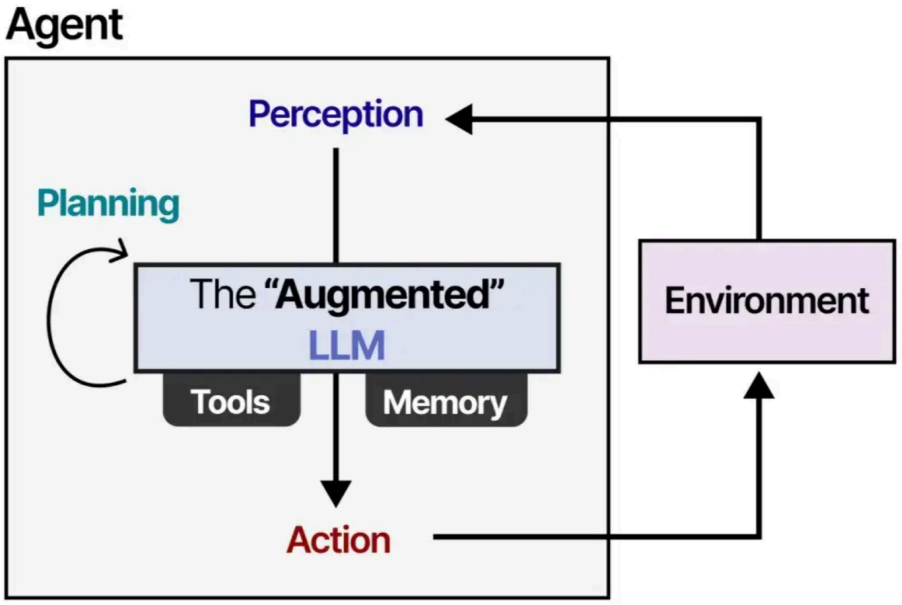
例如，一个复杂的查询可能需要先调用搜索工具获取信息，再调用计算工具进行分析，最后调用输出工具生成报告。如果LLM无法智能地规划这些步骤、判断工具之间的依赖关系或动态调整调用顺序，就容易出现低效、错误或无法完成任务的情况。

1.1.2 解决方案

为了让LLM真正像人类一样思考和做事，我们需要进一步提升其对Tools的指挥与调用能力。这意味着赋予LLM更高的智能层级：不仅要学会“使用”工具，更要学会“规划”“协调”和“优化”工具的使用过程。

人类在解决问题时，会自然而然地进行多步推理、预判潜在障碍并灵活切换策略；同样，理想的LLM应能够自主构建工具调用流程、处理多工具间的交互逻辑，并在执行中实时反思和调整。这种智能指挥机制，正是从“遥控机械臂”向“自主肢体”演进的关键一步。它将使LLM在面对复杂、开放性任务时，展现出更接近人类的思考深度和执行效率。

1.2 Agent构成



1.2.1 组成构件

要让LLM真正实现对Tools的智能指挥，我们需要先理解智能代理（Agent）的核心组成构件。Agent可以看作是一个完整的"智能主体"，它由三个基础部分协同工作，共同构成其决策与执行能力。

- **LLM（大型语言模型）**：Agent的"大脑"和决策中心。它负责理解任务、进行推理和生成最终输出，就像人类思考问题时的核心认知系统。
- **Tools（工具）**：Agent的"手脚"和执行能力。它赋予Agent调用外部函数、API或资源的实际操作能力，使其能够完成单纯靠语言模型无法直接处理的任务，例如搜索信息、执行代码或访问数据库。
- **Memory（记忆机制）**：Agent的"经验库"和上下文支撑。它负责存储历史对话、工具调用结果、中间推理步骤以及过往任务经验，帮助Agent保持任务连续性，避免重复工作，并在后续决策中参考历史信息。

这三个构件相互配合：LLM提供智慧，Tools提供行动力，Memory提供持续性，共同让Agent从"被动响应"转变为"主动思考"。

1.2.2 Environment

Environment（环境）是Agent所处的外部世界。它不仅仅是静态的背景，而是Agent必须持续交互的动态系统，包括当前的任务目标、可用的Tools集合、外部数据源的状态、网络条件、用户输入以及任何可能实时变化的因素。

例如，当Agent处理一个"帮我规划一次旅行"的复杂任务时，Environment中可能包含实时天气API、航班查询工具、地图服务以及用户偏好等资源。Environment会不断向Agent提供信息，同时也会受到Agent行动的影响（如调用工具后数据发生更新）。只有深入理解并适应Environment，Agent才能做出符合实际的决策。

1.2.3 行为

Agent与Environment的交互是通过一套完整的行为循环实现的，这一循环由Perception（感知）、Planning（规划）和Action（行动）三个环节紧密连接，形成持续迭代的智能过程。

- **Perception（感知）**：Agent首先通过工具调用或观察机制，实时获取Environment的反馈和当前状态信息。例如，调用搜索工具后，Agent会分析返回的结果，判断哪些信息可用、哪些还需要补充。
- **Planning（规划）**：在获得感知信息的基础上，Agent制定整体行动策略和具体步骤。它会进行多步推

理、选择合适的Tools、安排调用顺序，并预判可能的风险。例如，规划"先查询天气、再预订机票、最后生成行程表"的完整流程。

- **Action（行动）**：Agent将规划转化为实际执行，调用具体Tools并将结果反馈回Environment。同时，新的反馈又会进入下一轮Perception，形成闭环。

通过这种"感知—规划—行动"的循环，Agent能够像人类一样灵活应对复杂环境，不断调整策略，直至任务完成。这一行为机制正是Agent区别于普通LLM的核心所在，也为后续更高级的Agent架构奠定了基础。

1.2.4 Agent是什么

综合以上内容，我们可以对**Agent（智能代理）**做出如下清晰的定义：

Agent是LLM在Tools和Memory的支持下，形成的能够自主感知环境、规划行动并执行任务的完整智能主体。

它不再是单纯依赖预训练知识生成文本的"语言模型"，而是一个具备"大脑（LLM）+手脚（Tools）+经验库（Memory）"的自主决策系统。Agent通过与Environment（环境）的持续交互——先通过Perception感知当前状态，再通过Planning制定策略，最后通过Action执行操作——形成一个闭环的智能循环。这一循环使Agent能够像人类一样思考、适应和行动，灵活应对复杂、动态的任务场景。

简单来说，**Agent = LLM + Tools + Memory + 智能行为循环**。它标志着LLM从"被动响应者"向"主动问题解决者"的根本转变，也是后续章节中各种高级Agent架构（如ReAct、Plan-and-Execute等）的共同基础。

1.3 一个简单的示例

1.3.1 项目背景

我们实现一个简单但完整的智能体（Agent），它能自动完成：

- 读取本地文本文件
- 将文件内容从中文翻译成英文
- 将翻译结果保存到新的本地文件

这个 Agent 由以下部分组成：

- **LLM（大语言模型）**：负责实际翻译，并决定下一步调用哪个工具。
- **Tools（工具）**：两个真实函数——`read_file`（读文件）、`save_file`（写文件）。
- **感知—规划—行动循环**：Agent 通过这个循环与本地文件系统交互，一步一步完成任务。

这里我们先模拟 LLM 的感知和决策过程，主要了解一下Agent的整体结构。

1.3.2 代码示例

```
import json
from langchain.tools import tool
from langchain_core.runnables import Runnable

@tool
def load_file(filename: str) -> str:
    """加载本地文件内容，参数 filename 为文件路径"""
    with open(filename, 'r', encoding='utf-8') as f:
```

```
        return f.read()

@tool
def save_file(filename: str, content: str) -> str:
    """将内容保存到本地文件，参数 filename 为文件路径，content 为要保存的文本"""
    with open(filename, 'w', encoding='utf-8') as f:
        f.write(content)
    return f"已保存到 {filename}"

class SimpleAgent(Runnable):
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = {tool.name: tool for tool in tools}

    def invoke(self, input: dict, config=None):
        # 模拟决策：生成 JSON 指令
        load_cmd = json.dumps({"tool": "load_file", "args": {"filename": input["file_path"]}})
        cmd = json.loads(load_cmd)
        content = self.tools[cmd["tool"]].invoke(cmd["args"])

        if input["task"] == "translate":
            result = self.llm.invoke(f"翻译成英文：\n{content}")
        elif input["task"] == "classify":
            result = self.llm.invoke(f"分类：\n{content}")
        else:
            result = content

        save_cmd = json.dumps({"tool": "save_file", "args": {"filename": "output.txt", "content": result.content}})
        cmd = json.loads(save_cmd)
        self.tools[cmd["tool"]].invoke(cmd["args"])

        return {"result": "处理完成"}
```

调用示例：

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
load_dotenv()

llm = ChatOpenAI(model="deepseek-v4-flash")
tools = [load_file, save_file]
agent = SimpleAgent(llm, tools)
response = agent.invoke({
        "file_path": "./data.txt",
        "task": "translate"
    })
print(response)
```

1.3.3 Agent的实际使用

在上面的代码里面，我们完成了 Agent 的基础实现，不过并没有完全让大模型自己做主实现，而是模拟了决策过程。那么如果想要完全让大模型自己决策，应该如何操作?

（1）一般流程

定义Tools（使用@tool装饰器或Tool类）。

选择LLM（ChatOpenAI、ChatAnthropic等）。

使用创建函数构建Agent。

调用Agent:

- agent.invoke({"messages": [...]}) —— 同步执行
- agent.stream(...) —— 流式输出（推荐用于生产）

Agent会自动维护消息历史（Memory），并通过LangGraph的检查点（checkpointer）实现多轮对话持久化。

（2）创建 agent 的写法

```python
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

@tool
def my_tool(arg: str) -> str:
    """工具描述"""
    ...

llm = ChatOpenAI(model="deepseek-chat")   # 或其他模型

agent = create_agent(
    model=llm,
    tools=[my_tool],
    system_prompt="你是一个有帮助的助手。",   # 可选
    # middleware=...   # 高级动态控制
)

# 执行
result = agent.invoke({
    "messages": [{"role": "user", "content": "请帮我完成以下任务..."}]
})
```

我们发现，整个流程，会变得特别简单。

（3）实际使用

```python
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# ====================== 1. 使用 @tool 装饰器定义工具 ======================
```

```python
@tool
def load_file(filename: str) -> str:
    """加载本地文件内容，参数 filename 为文件路径"""
    with open(filename, 'r', encoding='utf-8') as f:
        return f.read()

@tool
def save_file(filename: str, content: str) -> str:
    """将内容保存到本地文件，参数 filename 为文件路径，content 为要保存的文本"""
    with open(filename, 'w', encoding='utf-8') as f:
        f.write(content)
    return f"已保存到 {filename}"

tools = [load_file, save_file]

# ====================== 2. 配置LLM   ======================
llm = ChatOpenAI(
    model="deepseek-chat"
)

# ====================== 3. 使用 create_agent 创建标准Agent ======================
agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt="""你是一个高效的文件处理Agent。
任务流程：
1. 使用 load_file 工具读取指定文件内容；
2. 根据用户要求，对内容进行“翻译成英文”或“分类”处理；
3. 使用 save_file 工具将处理后的结果保存到 output.txt；
4. 最后输出简洁的任务完成总结。

请严格使用工具完成每个步骤。"""
)
```

(4) 实际调用

```python
# 准备测试文件
user_task = """
帮我读取文件"./data.txt"，翻译成英文，并把内容保存到本地，路径名字为 './translated_data.txt'
"""
result = agent.invoke({
        "messages": [HumanMessage(content=user_task)]
    })

print("\n✅ Agent任务执行完成！")
print("最终输出：", result["messages"][-1].content)
```

输出结果：

```
✅ Agent任务执行完成！
最终输出： 任务已完成！以下是总结：

## ✅ 任务完成总结

| 步骤 | 状态 |
|------|------|
| 📁 读取文件 `./data.txt` | ✅ 成功（中文内容，约1700字） |
| 🌐 翻译成英文 | ✅ 完成（全文翻译） |
| 💾 保存到 `./translated_data.txt` | ✅ 成功 |

已成功将关于**人工智能**的中文文章完整翻译为英文，并保存到 `./translated_data.txt` 文件中。
```

（5）运行结果与分析

当运行上述代码后，Agent会**完全自主**完成整个流程：

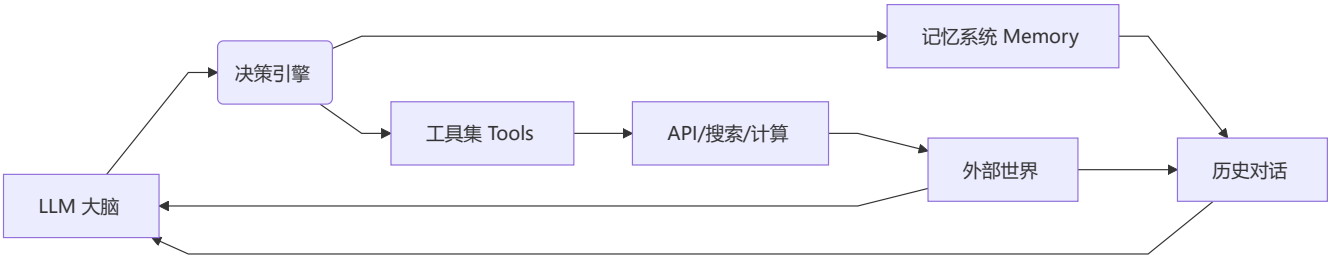
1. **Perception（感知）**：自动调用 load_file 工具读取 input.txt
2. **Planning（规划）**：LLM 思考任务要求，决定先读取、再翻译、最后保存
3. **Action（行动）**：自动调用 save_file 工具把翻译结果写入 output.txt
4. **Final Answer**：输出简洁的任务完成总结

核心意义：

- LLM 不再需要人工一步步写 if-else 或 JSON 硬编码，而是**真正像人类一样思考、规划、调用工具。**
- 代码量大幅减少，可读性与可维护性显著提升，完全符合2026年LangChain官方最佳实践。

2.Agent 深化理解

在掌握了Agent的基本搭建方法之后，我们需要进一步理解Agent在整个LLM生态中的定位，以及其内部实现的一些关键细节。本章将从“Agent与其他组件的区别”入手，用来区分不同概念，再深入讲解 create_agent 在实际运行中的 Memory 和 Planning等核心机制。



2.1 与其他组件的区别

Agent并不是孤立存在的概念，它与Chain、RAG以及其他常见AI组件有着本质差异。理解这些区别，有助于我们在实际项目中正确选择合适的技术方案。

2.1.1 Agent与Chain的区别

Chain（链） 是一种**固定流程**的组合方式。例如，一个典型的Chain可能依次执行"提示模板 → LLM调用 → 输出解析"三个固定步骤，整个过程是预先写死的、线性执行的。

Agent与Chain的核心区别在于"自主决策能力"：

- Chain：流程固定，无法根据中间结果动态调整路径。
- Agent：具备动态决策能力，可以根据当前环境和工具返回结果，自主决定下一步是继续调用工具、修改计划，还是直接输出答案。

简单来说，**Chain是"按剧本演戏"**，而**Agent是"即兴表演"。**当任务步骤明确且不需要灵活调整时，使用Chain更简洁；当任务复杂、需要工具动态调度时，Agent则更合适。

2.1.2 Agent与RAG的区别

RAG（Retrieval-Augmented Generation，检索增强生成） 的核心是通过检索外部知识库来补充LLM的知识不足，本质上是"检索 → 增强提示 → 生成"的单向流程。

Agent与RAG的本质区别在于"主动性与工具使用方式"：

- RAG：主要依赖被动检索（通常只有向量检索工具），目标是让LLM"知道更多信息"。
- Agent：可以主动调用多种Tools（搜索、计算、文件操作、API等），并通过多步规划和迭代来完成复杂任务。

换句话说，**RAG是"给LLM提供参考资料"**，而**Agent是"让LLM像助手一样主动工作"。**RAG适合知识密集型问答，Agent则适合需要多工具协作、规划和执行的任务。

2.1.3 Agent与其他常见AI组件的区别

除了Chain和RAG，Agent还与以下两种常见做法有明显不同：

- **与普通LLM的区别**： 普通LLM仅依赖提示生成文本，无法主动调用外部工具或获取实时信息。Agent则在LLM的基础上增加了Tools和自主循环，使其具备了"动手做事"的能力。
- **与纯Tool Calling / Function Calling的区别**： 纯Tool Calling是指LLM一次调用一个工具就结束。Agent则会**持续循环**：调用工具 → 观察结果 → 继续思考 → 再次调用工具，直到任务真正完成。这种"多轮自主循环"正是Agent区别于单次Tool Calling的关键。

总结：Agent的核心优势在于**动态规划 + 多工具协作 + 闭环执行**，它将LLM从"会说话"提升到了"会做事"。

2.2 Agent的实现细节

理解了Agent与其他组件的区别后，我们再来看看 create_agent 在实际运行时的一些内部机制。

2.2.1 create_agent 里面的 Memory

在Agent运行过程中，**Memory（记忆）** 负责保存对话历史、工具调用结果以及中间思考步骤，确保Agent在多轮交互中保持上下文连贯。

使用 create_agent 创建的Agent，**Memory是自动管理的。**不需要额外导入任何传统的Memory类，也不需要手动编写代码去维护历史记录。LangChain内部已经为Agent内置了完整的记忆机制，开箱即用。

这意味着：

- Agent会自动记住前面调用了哪些工具、得到了什么结果。
- 在多轮对话或复杂任务中，Agent不会“忘记”之前的步骤。
- 开发者只需专注于任务描述和工具定义，Memory部分几乎无需额外关注。

> “在现代Agent开发中，Memory已经不再需要我们手动去配置。create_agent 会自动帮我们处理好这一切，让我们把精力集中在任务逻辑和工具定义上。”

2.2.2 create_agent 里面的 Planning 等机制

create_agent 内部默认采用的是 **ReAct（Reason + Act）** 循环。在这个循环中，**Planning（规划）** 是LLM每次思考的核心环节。

具体过程如下：

- LLM 接收当前任务和历史信息后，先进行 **Thought（思考）**，制定下一步计划。
- 如果需要工具，则输出 **Action（行动）**（工具名称和参数）。
- 工具执行后返回 **Observation（观察结果）**。
- LLM 再次思考、规划，决定是继续调用工具还是输出最终答案。

这种规划机制完全由 **system_prompt**（系统提示）来引导。你可以通过精心设计system_prompt，来让Agent的规划更严谨、更结构化，从而适应不同类型的任务。

总结：create_agent 把Planning、Memory、工具调度等细节全部封装在内部，开发者只需提供合适的提示和工具，即可获得高效的Agent能力。

2.3 Agent的历史演进

为了帮助同学们更好地理解当前 `create_agent` 的设计理念，我们有必要系统回顾一下LangChain中Agent实现方式的历史发展过程。这段演进历史不仅能让我们看到技术优化的轨迹，也能避免在实际开发中误用过时的API。

2.3.1 0.1版本移除的initial创建方式

在 LangChain 最早的版本（0.1及之前）中，最主要的Agent创建方式是 `initialize_agent` 函数。

这个函数的设计初衷是让开发者能够快速初始化一个Agent，只需传入LLM、Tools和Memory等参数即可使用。它提供了较为简单的接口，适合初学者上手。然而，随着LangChain功能的不断丰富，`initialize_agent` 逐渐暴露出以下局限：

- API设计较为宽松，导致配置复杂时容易出错；
- 对新特性（如高级工具调用、结构化输出）的支持不足；
- 内部实现与后续的ReAct循环结合不够紧密。

因此，在LangChain 0.1版本的重大更新中，`initialize_agent` **被正式移除**，不再作为推荐用法。官方明确建议所有新项目都转向更现代的创建函数。这一变化标志着LangChain从“快速上手”向“生产级标准化”方向的转变。

2.3.2 2025年仍广泛使用的多个创建方式

在2025年（尤其是上半年到10月前后），以下几种Agent创建函数仍然非常好用且被广泛采用，是当时的主流选择：

函数名称	适用场景	核心特点
`create_tool_calling_agent()`	支持工具调用的模型	自动处理工具描述注入 结构化工具调用 多步骤执行
`create_react_agent()`	通用代理场景 (包括不支持工具调用的模型)	基于ReAct框架 文本格式工具调用 更灵活提示控制
`create_openai_functions_agent()`	专为OpenAI函数调用优化	OpenAI特定格式处理 自动函数描述生成 简化OpenAI集成
`create_self_ask_with_search_agent()`	问答+搜索场景	自问自答模式 聚焦搜索工具集成 清晰中间步骤
`create_structured_chat_agent()`	多参数工具场景	结构化工具参数 复杂工具调用 类型安全

LangChain在演进过程中仍保留了他们，目前它们被标记为 **Legacy（遗留）** 或 **Deprecated（已弃用）**，仅用于兼容历史代码。

这些函数在2025年的特点：

- 使用简单、文档丰富、社区案例众多；
- 能够很好地满足大多数教学和中小型项目的需求；
- 许多开发者在2025年整个年度都在稳定使用它们。

直到2025年底至2026年初，随着LangChain v1.0和LangGraph v1.0的发布，官方才开始对这些函数进行**正式标记为Deprecated（已弃用）**，并强烈推荐迁移到新的 create_agent。

2.3.3 传统的Memory模块

与Agent创建函数的演进同步，早期的 LangChain 还提供了独立的 **Memory模块**，用于手动管理Agent的记忆能力。常见的传统Memory类包括：

- `ConversationBufferMemory`：最基础的记忆方式，完整保存全部对话历史。
- `ConversationBufferWindowMemory`：仅保留最近N轮对话，适合控制上下文长度。
- `ConversationSummaryMemory`：对历史对话进行摘要压缩，节省Token消耗。
- `ConversationSummaryBufferMemory`：结合了摘要和窗口两种机制的混合版本。
- 其他特殊用途的Memory（如实体记忆、知识图谱记忆等）。

这些Memory模块在2025年的使用情况：

- 需要开发者手动创建实例并显式传递给Agent；
- 配置相对繁琐，但当时是实现多轮对话记忆的“标准做法”；
- 在2025年的项目中被广泛使用。

随着 `create_agent` 的推出，**这些传统的Memory模块已不再是主流。**新架构中Memory被内置到Agent的运行机制中，开发者无需手动导入或配置任何Memory类，极大简化了开发流程。

总结：

LangChain的Agent实现从早期“initialize_agent + 手动Memory”的碎片化模式，逐步演进到2026年统一的 `create_agent` 标准。这一演进过程体现了框架在**简洁性、可维护性与生产级能力**上的持续优化。了解这段历史，能帮助我们在实际开发中避免使用过时API，直接采用当前最佳实践。

2.3.4 AgentExecutor（通用执行器）

在2025年的LangChain生态中，**AgentExecutor（通用执行器）** 可以说是当时Agent实现里最核心、最不可或缺的组件之一，被广大开发者亲切地称为“Agent的灵魂引擎”。

AgentExecutor的主要职责是：

- 负责整个Agent的运行循环（ReAct 循环或其他模式）；
- 管理工具调用、Observation返回、Memory更新以及错误处理；
- 控制verbose日志输出、流式处理和最终结果解析。

典型使用方式（2025年标准写法）：

```
from langchain.agents import AgentExecutor

agent_executor = AgentExecutor(
    agent=agent,          # 通过create_react_agent等创建的agent
    tools=tools,
    verbose=True,         # 显示详细思考过程
    handle_parsing_errors=True
)

result = agent_executor.invoke({"input": "你的任务描述..."})
```

它就像一位“忠实的执行官”，把LLM的思考结果转化为真正的行动，并把每一步的结果可靠地反馈回去。没有AgentExecutor，当时的Agent就无法真正“跑起来”。

为什么说它是个“枭雄”?

- 在2025年及之前，几乎所有实际项目中的Agent都必须通过AgentExecutor来驱动；
- 它承载了大量生产实践，社区积累了海量使用经验和最佳实践；
- 即使现在被 `create_agent` 完全内置取代，它依然是 LangChain Agent发展史上浓墨重彩的一笔。

当前现状：

随着2026年 `create_agent` 的推出，AgentExecutor已不再需要开发者手动创建和配置，其核心功能已被官方新架构完整内置。官方不再推荐在新建项目中使用独立的AgentExecutor，但了解它的存在，仍有助于我们阅读和迁移2025年的历史代码。

2.3.5 其他被移除或弃用的重要组件

除了前面介绍的 initialize_agent、create_react_agent 等主流创建函数，以及 AgentExecutor 和传统 Memory 模块之外，LangChain 在演进过程中还对以下组件进行了移除或标记为弃用：

早期特定 Agent 类： 如 ZeroShotAgent、ConversationalAgent、JSONAgent 等传统 Agent 类型。这些类曾是早期实现自定义 Agent 的主要方式，但随着 create_agent 的统一，现在已被正式弃用，仅在 langchain-classic 包中保留用于兼容旧代码。

部分旧的工具套件（Agent Toolkits）： 早期 langchain.agents.agent_toolkits 中的一些工具集合被标记为 DEPRECATED_AGENTS，不再推荐直接使用。官方建议通过自定义 @tool 或新工具接口来替代。

其他辅助组件：

- 部分旧的输出解析器（Output Parser）和提示模板（Prompt Template）在 Agent 场景下已被新架构替代；
- 早期的一些检索相关组件（如 RetrievalQA）也因与新 Agent 体系不兼容而被弃用（虽不完全属于 Agent，但常与 Agent 结合使用）。

这些被移除或弃用组件的共同归宿： 目前所有历史遗留代码都被统一收录到 **langchain-classic** 包中。这个包的存在就是为了让2025年及之前的旧项目能够继续运行，而新项目则完全不需要再引入它。

总结： LangChain 通过引入 create_agent 并将大量旧组件移入 langchain-classic，实现了从“功能丰富但碎片化”到“简洁统一、生产级”的转变。了解这些被移除或弃用的内容，能帮助我们在阅读历史代码时快速定位问题，同时避免在新项目中误用旧 API。

LangChain Agent 的演进史，正如“大争之世”一般，早期各种创建方式和组件百家争鸣、各自为政。而2026年的create_agent，犹如秦皇一扫六合，结束了诸侯割据的局面，实现了真正的统一与雄霸天下。从此，开发者只需掌握一个标准函数，便能驾驭强大、简洁且生产级的Agent能力。这一统一，不仅大幅降低了学习和维护成本，更为后续更复杂的Agent应用奠定了坚实基础。

2.4 Agent 实际应用细节

通过前面的学习，同学们已经掌握了Agent的理论基础、与其他组件的区别以及历史演进。接下来，我们重点讲解 `create_agent` 在实际开发中常用的几个关键细节。这些细节虽然看似“琐碎”，却是让Agent从“能跑”变成“好用、稳用”的关键所在。

2.4.1 开启详细思考过程

在实际调试中，我们最需要看到Agent的**完整思考链**（Thought → Action → Observation → Final Answer）。

（1）实现方式

使用 `create_agent` 时，可以通过以下方式，开启详细日志：

在创建Agent时设置 debug=True

```
agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt=...,
    debug=True          # 开启详细日志
)
```

开启后，控制台会自动打印每个节点的执行细节、状态更新和流程转换，非常适合开发调试阶段使用。

（2）**推荐做法**：

- 开发/调试阶段：开启详细日志，清晰观察Agent每一步的决策过程。
- 生产环境：关闭详细输出，避免日志过多。

通过观察思考过程，我们可以快速定位Agent规划不合理、工具调用错误等问题。

（3）练习项目：

现在我们使用前面获取的天气查询 API，让大模型完成旅游攻略。

```
import os
import requests
from datetime import datetime
import pytz
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# ====================== 1. 真实天气Tool（保持你原来的API） ======================
@tool
def get_weather(city: str) -> str:
    """
    获取指定城市的实时天气信息（使用心知天气API）
    参数:
        city: 城市名称（支持中文或拼音），例如"北京"、"上海"
    返回:
        格式化的天气信息字符串，如果失败返回"错误信息:"开头
    """
    api_key = os.getenv("SENSIVERSE_API_KEY")
    if not api_key:
        return "错误信息: 未设置 SENSIVERSE_API_KEY 环境变量"

    base_url = "https://api.seniverse.com/v3/weather/now.json"
    params = {
        "key": api_key,
        "location": city,
        "language": "zh-Hans",
        "unit": "c"
    }
    try:
        response = requests.get(base_url, params=params, timeout=10)
        response.raise_for_status()
```

```
        data = response.json()
        if "results" in data and data["results"]:
            result = data["results"][0]
            return (
                f"地点: {result['location']['name']}\n"
                f"天气: {result['now']['text']}\n"
                f"温度: {result['now']['temperature']}°C\n"
                f"更新时间: {result['last_update']}"
            )
        return "错误信息: 未找到该城市的天气信息"
    except Exception as e:
        return f"错误信息: {str(e)}"


# ====================== 2. 获取当前时间 ======================
@tool
def get_current_time(city: str = "北京") -> str:
    """获取指定城市的当前日期、时间和星期（默认北京时间）"""
    tz = pytz.timezone("Asia/Shanghai")  # 中国统一使用东八区
    now = datetime.now(tz)
    return now.strftime(f"%Y年%m月%d日 %H:%M 星期%A（{city}当地时间）")


# ====================== 3. 获取城市景点 ======================
@tool
def get_city_attractions(city: str) -> str:
    """获取指定城市的主要旅游景点（返回3-5个热门景点）"""
    attractions = {
        "北京": "故宫、天安门、长城、颐和园、鸟巢",
        "上海": "外滩、东方明珠、豫园、南京路、迪士尼",
        "广州": "广州塔、长隆野生动物世界、陈家祠、珠江夜游",
        "深圳": "世界之窗、欢乐谷、莲花山公园、大梅沙海滨",
        "杭州": "西湖、灵隐寺、雷峰塔、千岛湖",
    }
    return attractions.get(city, f"{city}暂无预设景点，可参考当地旅游官网")

tools = [get_weather, get_current_time, get_city_attractions]

# ====================== 4. 创建Agent ======================
llm = ChatOpenAI(model="deepseek-chat")

agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt="""你是一个专业、细心的一日游规划助手。
用户输入一个城市后，请严格按照以下顺序执行：
1. 调用 get_weather 工具获取实时天气；
2. 调用 get_current_time 工具获取当前时间；
3. 调用 get_city_attractions 工具获取该城市主要景点；
4. 综合以上信息，为用户生成一个**简单的一日旅游攻略**（包括：上午、中午、下午、晚上建议行程、注意事项）。
回复要简洁、实用、友好，最后以“祝你玩得开心！”结束。""",
    debug=True
```

```
)
```

（4）运行示例

```
user_input = "帮我规划一下今天北京的一日游行程"

print("智能一日游规划Agent启动...\n")

result = agent.invoke({
        "messages": [HumanMessage(content=user_input)]
})

print("\n✅ Agent生成的一日游攻略：")
print(result["messages"][-1].content)
```

2.4.2 错误处理与容错机制

（1）使用说明

Agent在实际运行中难免会遇到工具执行失败、LLM输出格式错误或网络异常等情况。`create_agent` 内置了较为完善的错误处理能力，我们可以通过以下方式进一步增强：

```
agent_executor = create_agent(...)  # create_agent返回的可直接使用

# 推荐配置
result = agent.invoke(
    {"messages": [...]},
    config={
        "configurable": {
            "handle_parsing_errors": True,    # 自动处理解析错误
            "max_iterations": 15,             # 防止无限循环
        }
    }
)
```

（2）**实用建议**：

- 设置合理的 `max_iterations` 防止Agent陷入死循环。
- 对于关键任务，可在 `system_prompt` 中明确要求Agent在出错时输出友好提示并尝试恢复。
- 生产环境中建议结合try-except捕获高层异常，并记录日志。

（3）示例练习

这里我们构建一个简单的求购物车商品总价小练习，故意把tool写得不鲁棒。

```
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

```

```python
# ====================== 1. Tool（增加异常捕获，制造教学Bug） ======================
@tool
def calculate_total_price(items: list) -> float:
    """
    直接对传入的列表进行求和，不要做任何转换
    注意：本工具不做任何类型转换或解析，请确保传入的是纯数字列表。
    """
    try:
        # 故意最简单，只调用sum()，制造Bug场景
        return sum(items)
    except Exception as e:
        # 关键修复：把异常转化为字符串返回给Agent
        return f"工具执行错误: {type(e).__name__} - {str(e)}"


tools = [calculate_total_price]

# ====================== 2. 创建Agent ======================
llm = ChatOpenAI(model="deepseek-chat", temperature=0)

agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt="""你是一个购物车计算助手。
用户会给你一段文本，其中包含一个数字列表（可能是字符串形式的列表或元组）。
你的任务是：
1. 从用户输入中提取出那个数字列表；
2. 使用 calculate_total_price 工具对提取出的列表进行计算；
3. 如果工具返回“工具执行错误”信息，请仔细阅读错误内容，分析原因，然后修正后重新调用工具；
4. 最终给出清晰的总价结果。""",
    debug = True  # 保持 debug 为 True
)

# ====================== 3. 运行示例 ======================

# 用户只输入一个字符串形式的列表（模拟真实输入）
user_task = "请帮我计算以下商品的总价：[99, 199, '299', 50]"
# 也可以测试元组形式：
# user_task = "请帮我计算以下商品的总价：(10, 20, 30, 40)"

print("🚀 购物车总价计算Agent启动...\n")
print("用户输入：", user_task)
print("-" * 80)

result = agent.invoke(
        {"messages": [HumanMessage(content=user_task)]},
        config={
            "configurable": {
                "handle_parsing_errors": True,
                "max_iterations": 15,
            }
        }
    )
```

```
print("\n✅ Agent最终回复：")
print(result["messages"][-1].content)
```

2.4.3 流式输出（Streaming）

（1）使用说明

实际应用中，用户通常希望看到Agent**实时思考和输出**，而不是等全部执行完才返回结果。`create_agent` 支持流式输出：

```
for chunk in agent.stream(
    {"messages": [HumanMessage(content=user_task)]}
):
    print(chunk)
```

（2）**适用场景**：

- 交互式聊天机器人
- 长任务处理（让用户看到进度）
- 提升用户体验

（3）修改前面例子

在 2.4.2 里面，我们看到的输出对于用户体验并不好，我们可以使用流式输出，让agent把具体过程输出出来。

```
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# ====================== 1. 严格Bug版 Tool ======================
@tool
def calculate_total_price(items: list) -> float:
    """
    直接对传入的列表进行求和，不要做任何转换
    注意：本工具不做任何类型转换或解析，请确保传入的是纯数字列表。
    """
    try:
        # 故意最简单，只调用sum()，制造Bug场景
        return sum(items)
    except Exception as e:
        # 关键修复：把异常转化为字符串返回给Agent
        return f"工具执行错误: {type(e).__name__} - {str(e)}"


tools = [calculate_total_price]

# ====================== 2. 创建Agent ======================
llm = ChatOpenAI(model="deepseek-chat", temperature=0)

```

```
agent = create_agent(
    model=llm,
    tools=tools,
    debug=False,
    system_prompt="""你是一个购物车计算助手。
用户输入中包含一个数字列表。
你的任务是：
1. 从用户输入中提取数字列表（可能是字符串形式）；
2. 直接调用 calculate_total_price 工具；
3. 如果工具返回“工具执行错误”，请**明确说出**错误原因，然后修正列表后重新调用工具；
4. 最终给出总价结果。"""
)

# ====================== 3. 清晰打印每一步（教学友好版） ======================
if __name__ == "__main__":
    user_task = "请帮我计算以下商品的总价：[99, 199, '299', 50]"

    print("购物车总价计算Agent启动（错误恢复演示）...\n")
    print(f"用户输入：{user_task}\n")
    print("=" * 90)

    # 最原始、最直接的打印方式（不再依赖 "messages" 判断）
    for chunk in agent.stream(
        {"messages": [HumanMessage(content=user_task)]},
        config={"configurable": {"handle_parsing_errors": True, "max_iterations": 15}}
    ):
        # 直接打印 chunk 中所有有 content 的消息
        print(chunk)

    print("\n✅ Agent任务完成！")
```

我们发现打印 chunk 的话，会显得特别的乱，我们直接按照结构，打印里面的信息：

```
for chunk in agent.stream(
        {"messages": [HumanMessage(content=user_task)]},
        config={"configurable": {"handle_parsing_errors": True, "max_iterations": 15}}
):
    for key,value in chunk.items():
        print(f"{key}:")
        print(value["messages"][0].content)
    print('=' * 80)
```

注意，这种写法的鲁棒性并不强，建议采用下面的写法：

```
for chunk in agent.stream(
        {"messages": [HumanMessage(content=user_task)]},
        config={"configurable": {"handle_parsing_errors": True, "max_iterations": 15}}
    ):
        # 直接打印 chunk 中所有有 content 的消息
        if isinstance(chunk, dict):
            for key, value in chunk.items():
                if isinstance(value, dict) and "messages" in value:
```

```
                    for msg in value["messages"]:
                        if hasattr(msg, "content") and msg.content:
                            print(msg.content.strip())
                            print("-" * 80)
                elif hasattr(value, "content") and value.content:
                    print(value.content.strip())
                    print("-" * 80)
```

2.4.4 Human-in-the-Loop（人工介入）

当Agent遇到无法自主决策的情况（如需要用户确认、敏感操作等）时，可以使用 **Human-in-the-Loop** 机制让人工介入：

```
# 在system_prompt中加入提示
system_prompt = """...
如果需要用户确认某些关键操作，请输出 "ASK_HUMAN: [你的问题]" 并停止执行。
"""

# 执行时可通过中断点实现人工介入（具体实现可结合前端或输入函数）
```

这一机制让Agent从“完全自主”变为“人机协作”，在企业级应用中尤为重要。

2.4.5 system_prompt的优化技巧

`create_agent` 的行为很大程度上由 `system_prompt` 决定。以下是几条实战经验：

- **明确角色**：开头写清楚Agent的身份（如“你是一个严谨的文件处理专家”）。
- **严格流程**：明确列出任务步骤（如“必须先load_file，再翻译，最后save_file”）。
- **输出格式要求**：要求Agent在最终答案前使用“Final Answer:”标记。
- **错误恢复**：加入“如果工具调用失败，请分析原因并尝试其他方案”。

一个精心设计的system_prompt，往往能让Agent的成功率提升30%以上。

3.Agent 的自主决策

3.1 问题概述

3.1.1 之前写法的问题

在前面编写 Agent 时，我们采用 “流程硬编码” 的方式来编写 `system_prompt`。典型代码如下：

```
agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt="""你是一个购物车计算助手。
用户会给你一段文本，其中包含一个数字列表（可能是字符串形式的列表或元组）。
你的任务是：
1. 从用户输入中提取出那个数字列表；
2. 使用 calculate_total_price 工具对提取出的列表进行计算；
3. 如果工具返回“工具执行错误”信息，请仔细阅读错误内容，分析原因，然后修正后重新调用工具；
4. 最终给出清晰的总价结果。""",
    debug=True
)
```

这种写法的主要特点（而非缺陷）：

- **提供明确的任务流程**：Agent 能清晰理解要完成的具体步骤，在简单场景下表现稳定。
- **推理模式相对固定**：主要体现“顺序执行 + 条件判断”的模式，适合结构清晰的任务。
- **在现代 LLM 下的表现**：得益于当前 LLM 强大的 tool-calling 能力，即使是这种流程导向的 prompt，Agent 也往往能顺利调用工具并完成计算。

然而，这种写法本质上**将一种特定的推理模式（顺序流程）硬编码进了 prompt**。如果我们希望 Agent 展现**更多样的推理模式**（例如主动思考、灵活决策、多轮迭代等），就需要通过调整 system_prompt 来实现。

之前写法本身是有效的起点，但它只代表了“提示词决定推理模式”中的一种情况。通过后续不同 prompt 的对比，我们可以直观看到提示词如何引导 Agent 进入不同的推理模式。

3.1.2 介绍提示词对agent的重要性

在 LangChain 1.0 之后，Agent 的构建方式已简化为 create_agent + system_prompt 的统一形式。此时：**system_prompt 成为决定 Agent 采用哪种推理模式的核心因素。**

即使现代 LLM 的 tool-calling 能力已经非常强大（不写任何详细 prompt 时，Agent 也能较好地调用工具），**提示词仍然是引导 Agent 进入特定推理模式的“开关”。**

（1）**提示词对 Agent 的核心价值在于**：

- **决定推理模式**

  同一个 Agent、同一套工具，不同的 system_prompt 可以让它采用完全不同的思考方式。

- **从基础到高级的平滑升级**

  我们可以通过**渐进式提示词实验**直观展示这一点（推荐课堂使用完全相同的测试输入）：

（2）**阶段1（基础提示词）**：

```
system_prompt="你是一个有帮助的助手。"
```

→ Agent 进入 基础 CoT（Chain of Thought） 模式，能自然地进行简单推理和工具调用。

（3）**阶段2（ReAct 提示词）**：

```
system_prompt="""请严格遵循 ReAct 模式：
Thought: 先思考当前用户输入和目标
Action: 需要工具就调用，否则直接回答
最终用 "Final Answer: " 开头。"""
```

→ Agent 切换到 ReAct（Reason-Act-Observe）推理模式，开始有意识地"思考 → 行动 → 观察"循环。

结论：提示词不是"必须写得很复杂才能让 Agent 工作"，而是"通过精心设计来主动选择和切换 Agent 所使用的推理模式"。

3.1.3 优化system_prompt与提示词工程的区别

虽然两者高度相关，但有清晰的区别，需要区分清楚才能真正掌握：

维度	优化 system_prompt	提示词工程（Prompt Engineering）	关系说明
定义	针对**特定 Agent** 的 system_prompt 优化	一整套系统化的提示词设计方法论	优化是"实战应用"，工程是"方法论"
范围	专注于当前任务的 system_prompt 编写	包含 CoT、ReAct、Few-shot、Tree of Thoughts、Reflexion 等多种通用技巧	优化是提示词工程在 LangChain Agent 中的具体实践
目标	让当前 Agent 具备自主决策、错误恢复等能力	让 LLM 在各种场景下产生更好、更可靠的输出	优化是工程方法在 Agent 场景下的落地
示例	"根据实际情况自主判断下一步行动"	"使用 ReAct 模式 + Few-shot 示例 + 反思机制"	优化通常会运用工程中的多种技巧
适用阶段	LangChain 1.0+ 的 create_agent 阶段	适用于所有 LLM 应用（Chat、RAG、Agent 等）	优化是工程的子集

关键教学点：

- **优化 system_prompt** 是"微观操作"：针对购物车计算助手这个具体场景，写出最有效的 system_prompt。
- **提示词工程** 是"宏观方法论"：学习 ReAct、Few-shot、Chain-of-Thought 等通用技巧后，再应用到 system_prompt 的优化中。
- 在本课程中，我们先掌握"优化 system_prompt"的实战技能（即本节内容），后续章节会深入学习更高级的提示词工程技巧（如 Few-shot、middleware 辅助等）。

小结：优化 system_prompt 是提升 Agent 自主决策能力最直接、最有效的方式，而提示词工程则是支撑这种优化的底层方法论。掌握两者区别后，学生就能从"会抄提示词"进化到"会设计智能 Agent"。

3.2 常见工具介绍

3.2.1 维基百科

在 Agent 课程中，我们现在使用 **Wikipedia** 作为知识检索工具。它内容权威、支持中文搜索、无需申请 API Key，非常适合教学演示和知识型任务。

（1）核心安装与导入

```
# 1. 安装（只需运行一次）
pip install wikipedia

# 2. 导入
import wikipedia
from langchain_core.tools import tool
```

（2）创建 Wikipedia 搜索工具（官方推荐写法）

```
@tool
def wikipedia_search(query: str) -> str:
    """使用维基百科API搜索信息（支持中文）"""
    try:
        # 设置语言为中文
        wikipedia.set_lang("zh")
        # 处理中文查询
        if "维基百科" in query:
            query = query.replace("维基百科", "").strip()
        # 搜索页面
        page = wikipedia.page(query, auto_suggest=False)
        return f"标题: {page.title}\n摘要: {page.summary[:500]}..."
    except wikipedia.DisambiguationError as e:
        options = e.options[:3]
        return f"模糊匹配: 请从以下选项中选择: {', '.join(options)}"
    except wikipedia.PageError:
        return "未找到相关信息"
    except Exception as e:
        return f"搜索错误: {str(e)}"
```

3.2.2 Baostock 股票数据工具介绍

在 Agent 课程中，我们使用 **Baostock** 作为**免费的股票数据 API**，专门用于获取中国 A 股（沪深市场）的历史行情、K 线数据、基本面信息等。它**无需注册、无需 API Key**，非常适合教学演示和量化分析类 Agent。

(1) 核心安装与导入

```
# 1. 安装（只需运行一次）
pip install baostock pandas

# 2. 导入
import baostock as bs
from langchain_core.tools import tool
```

(2) 创建 Baostock 股票查询工具（官方推荐写法）

```
@tool
def get_stock_history(code: str, start_date: str = "2024-01规则任务范式-01规则任务范式",
end_date: str = "2025-12-31", frequency: str = "d") -> str:
    """查询 A 股股票历史 K 线数据（日线/周线/月线）
    code 示例：sh.600519（贵州茅台）、sz.000001（平安银行）
    frequency: d=日线, w=周线, m=月线
    """
    try:
        # 必须先登录
        login_result = bs.login()
        if login_result.error_code != '0':
            return f"登录失败: {login_result.error_msg}"

        # 查询历史 K 线
        rs = bs.query_history_k_data_plus(
            code=code,
            fields="date,code,open,high,low,close,volume,amount,pctChg",
            start_date=start_date,
            end_date=end_date,
            frequency=frequency,
            adjustflag="3"   # 3=后复权（最常用）
        )

        # 登出
        bs.logout()

        if rs.error_code != '0':
            return f"查询失败: {rs.error_msg}"

        # 转为 DataFrame 并返回前 10 条 + 总结（避免输出过长）
        df = rs.get_data()
        summary = f"共获取 {len(df)} 条记录\n最近10条数据:\n{df.tail(10).to_string()}"
        return summary

    except Exception as e:
        return f"工具执行错误: {str(e)}"
```

最核心的一句话总结:

`get_stock_history` 工具让 Agent 可以直接输入股票代码（如 `sh.600519`）和日期范围，自动返回结构化的历史行情数据（pandas DataFrame 格式），非常适合股票分析、价格查询、投资决策类 Agent。

(3) 上手提示：

- 股票代码格式：沪市前缀 `sh.`，深市前缀 `sz.`（例如 `sh.600519` 是贵州茅台）
- 默认返回**后复权**数据，适合做回测和分析
- 输出是文本格式，Agent 可以轻松阅读并进一步计算/分析
- 完全免费、无调用次数限制，是目前教学中最推荐的 A 股数据工具

4. 常见 Agent 框架对比表

框架/模式名称	核心思想	起源	适用场景	关键优势
ReAct	推理-行动交替循环	普林斯顿大学 & Google Research (2022)	通用问题解决	平衡思考与行动，决策透明
Plan-and-Execute	先整体规划再分步执行	LangChain 2023（受 BabyAGI 启发）	复杂多步长任务	任务分解清晰，容错性强
OpenAI Function Calling	函数/工具调用驱动	OpenAI API 2023	API 集成与外部工具调用	原生支持、低延迟、结构化输出
Structured Chat	结构化对话 + 多参数工具	LangChain 2023	多参数复杂工具场景	参数验证强，适合复杂输入
Self-Ask	自问自答 + 后续搜索	Ofir Press et al. (2022)	事实型/组合式问答	精准检索，可解释性高
Chain-of-Thought	显式逐步推理链	Google (Wei et al.) 2022	逻辑/数学/复杂推理	大幅提升 LLM 推理能力

教学小贴士（可加在表格下方）：

- ReAct、CoT、Self-Ask 属于**提示词层面的推理技巧**（Prompting Techniques）。
- Plan-and-Execute、Structured Chat 属于**框架层面的 Agent 实现**（Agent Architectures）。
- OpenAI Function Calling 属于**底层工具调用能力**（Infrastructure）。
- 在 LangChain 1.0+ 中，这些模式大多通过 system_prompt 来实现切换，**提示词决定了 Agent 最终采用哪种推理模式。**

4.1 ReAct框架

4.1.1 框架起源，介绍

ReAct（全称 **Reasoning and Acting**，即"推理 + 行动"）是 Agent 领域最经典、最基础的推理框架之一。

- **提出时间**：2022年10月（论文初稿），2023年3月修订，最终发表于 ICLR 2023。
- **提出者**：普林斯顿大学 Shunyu Yao 等研究者，与 Google Research 团队合作。

- **论文标题**：《ReAct: Synergizing Reasoning and Acting in Language Models》（arXiv: 2210.03629）。
- **核心创新**：首次将 LLM 的"推理能力"（Chain-of-Thought）和"行动能力"（Tool Calling / 外部交互）**交替进行**，形成一个动态循环，让模型既能思考，又能通过工具获取真实信息，从而大幅提升复杂任务的解决能力。

在 LangChain 1.0+ 中，`create_agent` 默认采用的正是 ReAct 模式，开发者只需通过 `system_prompt` 即可轻松激活或强化这一框架。

4.1.2 框架特点

ReAct 框架的最大特点是**透明、可控、迭代式**的思考-行动循环，其核心流程为：

Thought（思考） → **Action（行动/工具调用）** → **Observation（观察结果）**，不断循环直到得出 Final Answer。

主要特点：

- **推理与行动高度协同**：思考帮助规划下一步行动，行动结果又反过来更新思考，避免单纯 CoT 的幻觉问题。
- **决策过程高度透明**：每一步都有明确的 Thought 记录，便于调试和解释（debug=True 时日志特别清晰）。
- **天然支持错误恢复**：遇到工具失败时，模型可通过下一轮 Thought 分析原因并修正。
- **少样本即可生效**：通常只需 1-2 个示例或清晰指令就能让 LLM 严格遵循 ReAct 模式。
- **与现代 LLM 完美适配**：当前 LLM 的 tool-calling 能力已极强，ReAct 能最大化发挥这一能力。

相比单纯的"一步到位"提示，ReAct 让 Agent 从"一次性回答"进化成"多轮迭代思考"的智能体。

4.1.3 框架写作模板

ReAct 的 system_prompt 写作模板非常简洁且标准化，推荐在 LangChain 1.0+ 中直接使用以下模板（可根据任务微调）：

```
system_prompt="""你是一个有帮助的助手，请严格遵循 ReAct 模式完成任务。

【ReAct 规则】
- Thought: 先思考当前用户输入、已知信息和下一步最优行动
- Action: 如果需要外部信息或计算，就调用对应工具；否则直接给出最终答案
- Observation: 仔细阅读工具返回的结果
- 重复 Thought → Action → Observation 循环，直到可以给出完整答案

最终答案请以 "Final Answer: " 开头，用清晰友好的语言回复用户。

现在开始处理用户输入，自主进行 ReAct 思考。"""
```

写作要点：

- 必须明确要求输出格式（Thought / Action / Observation）
- 强调"循环直到完成"
- 可根据具体工具名称在 prompt 中稍作补充说明

4.1.4 适用场景

ReAct 框架适用性极广，几乎是所有需要**工具调用 + 多步推理**任务的首选：

- 通用问题解决（搜索 + 计算 + 分析）
- 购物车计算、数据查询、价格对比等需要提取信息 + 调用工具的场景
- 需要与外部 API、数据库、搜索引擎交互的任务
- 希望决策过程透明、可解释、可调试的教学或生产环境

特别适合：输入不完全确定、可能需要多次工具调用或错误重试的场景。

4.1.5 适用示例

场景：购物车总价计算（延续本课程案例）

完整 system_prompt 示例（阶段2 ReAct 版本）：

```
agent_react = create_agent(
    model=llm,
    tools=tools,
    system_prompt="""你是一个智能旅行规划助手，请严格遵循 ReAct 模式。

【ReAct 规则】
- Thought: 先分析用户需求（时间、预算、偏好），决定下一步最优行动
- Action: 需要文化、历史、景点、交通等信息时调用 wikipedia_search 工具
- Observation: 仔细阅读工具返回的结果
- 重复循环，直到协调出完整、可行的旅行方案

最终用 "Final Answer: " 开头给出清晰、结构化的推荐方案（包含行程、日程、文化亮点、预算估算）。""",
    debug=True
)
```

4.2 Plan-and-Execute框架

4.2.1 框架起源，介绍

Plan-and-Execute（规划-执行框架）是 Agent 领域中经典的分层规划架构之一。

- **提出时间**：2023 年初
- **核心灵感来源**：受 “Plan-and-Solve Prompting” 论文（arXiv:2305.04091，2023）和 BabyAGI 项目启发
- **主要实现**：由 LangChain 框架正式落地并推广为一种 Agent 模式
- **核心创新**：将 LLM 的能力明确拆分为两个阶段 —— **先制定完整的高层计划（Plan）**，再 **严格按照计划逐条执行（Execute）**。每个执行步骤可以调用工具、获取观察结果，形成“先全局统筹、再局部落实”的工作流。

在 LangChain 1.0+ 中，Plan-and-Execute 不再是一个独立的 Agent 类，而是通过精心设计的 `system_prompt` 来实现，让开发者可以轻松在同一个 `create_agent` 中切换不同推理模式。

与 ReAct 的核心区别：ReAct 是“边思考边行动”的实时迭代循环，而 Plan-and-Execute 是“先整体规划、再分步执行”，更注重全局视角和任务结构化分解。

4.2.2 框架特点

Plan-and-Execute 框架的最大特点是**全局规划 + 结构化执行**，其核心流程为：

Plan（制定完整计划） → Execute（逐条执行 + 观察） → 完成所有步骤后输出 Final Answer。

主要特点：

- **全局视角优先**：在开始任何行动前，先输出完整、编号的执行计划，避免 ReAct 中可能出现的"走一步看一步"导致的遗漏或局部最优。
- **任务分解清晰**：擅长把复杂目标拆解成有序、可执行的子步骤，每个步骤目标明确。
- **执行过程可控**：每执行完一步，都会记录 Observation 并决定是否继续下一条计划，容错性较强。
- **适合长链条任务**：步骤之间存在明显依赖关系时表现优秀。
- **透明度高**：计划阶段和执行阶段分开，debug 日志非常清晰，便于学生观察和理解。

相比 ReAct，Plan-and-Execute 在规划阶段就"想得更远"，但灵活性稍弱（如果初始计划有偏差，后续调整成本较高）。

4.2.3 框架写作模板

Plan-and-Execute 的 system_prompt 写作模板重点在于**强制先输出 Plan，再 Execute**，推荐在 LangChain 1.0+ 中直接使用以下模板（可根据任务微调）：

```
system_prompt="""你是一个智能助手，请严格使用 Plan-and-Execute 模式完成任务。

【工作流程】
1. Plan 阶段：先制定一个完整、详细、编号的执行计划，列出所有必要步骤和每个步骤的目标。
2. Execute 阶段：严格按照计划，一步一步执行。每完成一步，就记录工具返回的结果（Observation），然后继续下一步。
3. 只有当所有计划步骤都完成后，才给出最终答案。

请先输出完整的 Plan（以 "Plan:" 开头），然后开始 Execute 过程。
最终答案请以 "Final Answer: " 开头，用清晰友好的语言回复用户。"""
```

写作要点：

- 必须强制要求"先 Plan 后 Execute"的顺序
- 计划要编号、清晰、可执行
- 可在 prompt 中补充具体工具名称，引导模型正确调用

4.2.4 适用场景

Plan-and-Execute 框架特别适合**复杂、多步骤、需要全局统筹**的任务：

- 长链条规划类任务（旅行行程、项目方案、学习计划）
- 步骤之间存在明显依赖关系或需要提前预算/统筹的场景
- 需要结构化输出（包含多个子模块）的分析型任务
- 希望 Agent 提前展示完整思路、便于人工审核的教学或生产环境

特别适合：输入目标明确但实现路径复杂、必须先分解再逐一落实的场景（与 ReAct 的"实时迭代"形成鲜明对比）。

4.2.5 适用示例

场景：智能旅行规划助手（与 ReAct 示例保持一致，便于课堂对比）

用户输入：

“帮我规划一个周末从北京去上海的旅行，预算控制在5000元以内，偏好文化体验和美食。”

完整 system_prompt 示例（Plan-and-Execute 版本）：

```
agent_plan_execute = create_agent(
    model=llm,
    tools=tools,
    system_prompt="""你是一个智能旅行规划助手，请严格使用 Plan-and-Execute 模式。

【工作流程】
1. Plan 阶段：先制定完整编号的执行计划（包含交通、文化景点、美食、预算等步骤）
2. Execute 阶段：严格按计划逐条执行，需要文化、历史、景点等信息时调用 wikipedia_search 工具
3. 所有步骤完成后给出最终方案

先输出完整的 Plan，然后开始 Execute。最终用 "Final Answer: " 开头给出结构化推荐方案。""",
    debug=True
)
```

教学价值：

可以直观看到 Plan-and-Execute 与 ReAct 的差异——Agent 不是“边走边想”，而是**先给出一份完整计划**，再有条不紊地逐条落实。通过同一个旅行规划任务的对比，学生能深刻理解不同推理模式如何影响 Agent 的决策过程，为后续自主决策模式的学习做好铺垫。

4.3 Self-Ask框架

4.3.1 框架起源，介绍

Self-Ask（自问自答框架）是 Agent 推理模式中一种经典的**问题驱动**（question-driven）方法。

- **提出时间**：2022 年
- **提出者**：Ofir Press（华盛顿大学）等研究者
- **核心论文**：《Self-Ask: A Simple Technique for Improving Multi-Hop Reasoning in LLMs》（2022）
- **核心创新**：让 LLM **自己生成后续问题（Follow-up Questions）**，然后通过工具回答这些问题，从而实现多跳（multi-hop）推理。模型不再一次性试图回答复杂问题，而是“先问自己一系列子问题，再逐一解决”，有效提升了事实性、组合式任务的准确性。

在 LangChain 1.0+ 中，Self-Ask 同样通过 `system_prompt` 来实现，无需额外框架代码，只需明确指示模型“先自问、再调用工具回答”即可激活该模式。

与前面框架的区别：ReAct 是“思考-行动-观察”通用循环，Plan-and-Execute 是“先整体规划、再逐条执行”，而 Self-Ask 是**纯问题驱动**的迭代方式，更注重“把大问题拆成一系列可回答的小问题”。

4.3.2 框架特点

Self-Ask 框架的最大特点是**自问驱动 + 逐步验证**，其核心流程为：

Self-Question（自问子问题）→ Action（调用工具回答该问题）→ Observation（记录答案）→ 生成下一个子问题，直到所有关键问题都得到解答后输出 Final Answer。

主要特点：

- **高度结构化的多跳推理**：擅长把复杂任务拆解成一系列逻辑连贯的子问题，避免一次性推理出错。
- **可解释性极强**：每一步都有明确的"Follow-up Question"，决策过程透明，便于学生理解和调试。
- **天然适合搜索型任务**：与搜索工具结合效果最佳，能有效减少幻觉（hallucination）。
- **迭代性强**：模型会根据前一个问题的 Observation 动态生成下一个问题，实现真正的"边问边答"。
- **实现简单**：只需在 prompt 中强调"先自问、再回答"的模式即可生效。

相比其他框架，Self-Ask 更"像学生做题"——先把问题拆成小问，再逐一解决，特别适合需要多轮信息收集和验证的场景。

4.3.3 框架写作模板

Self-Ask 的 system_prompt 写作模板重点在于**强制模型先自问、再调用工具**，推荐在 LangChain 1.0+ 中直接使用以下模板（可根据任务微调）：

```
system_prompt="""你是一个智能助手，请严格使用 Self-Ask 模式完成任务。

【Self-Ask 规则】
- 每一步先提出一个清晰的 Follow-up Question（自问子问题）
- 然后调用工具来回答这个子问题
- 记录工具返回的 Observation
- 根据当前已知答案，生成下一个 Follow-up Question
- 重复"自问 → 工具回答 → 观察"循环，直到所有必要问题都得到解答
- 只有当所有关键问题都解决后，才给出最终答案

请以 "Follow-up Question: " 开头提出子问题，最终答案以 "Final Answer: " 开头。"""
```

写作要点：

- 必须明确输出格式（Follow-up Question / Action / Observation）
- 强调"根据上一个答案生成下一个问题"
- 可在 prompt 中补充具体工具名称，引导模型正确使用

4.3.4 适用场景

Self-Ask 框架特别适合**事实型、多跳推理、需要多轮信息验证**的任务：

- 组合式问答（需要多个事实拼接才能得出结论）
- 研究型、搜索型任务（旅行规划、产品对比、背景调查等）
- 需要高准确性、避免幻觉的场景
- 希望 Agent 决策过程像"一步一步提问"一样清晰的教学或生产环境

特别适合：输入目标复杂、必须通过多个子问题逐步收集和协调信息才能完成的任务（与 ReAct 的"实时循环"和 Plan-and-Execute 的"先规划"形成鲜明对比）。

4.3.5 适用示例

场景：智能投资研究助手（复杂专业知识的推理）

用户输入："帮我分析贵州茅台的投资价值，包括公司背景、竞争优势、当前风险和未来机会。"

工具准备（同前，无需封装外部 API）：

完整 system_prompt 示例（Self-Ask 版本）：

```
agent_self_ask = create_agent(
    model=llm,
    tools=tools,
    system_prompt="""你是一个智能投资研究助手，请严格使用 Self-Ask 模式进行复杂专业知识推理。

【Self-Ask 规则】
- 每一步先提出一个 Follow-up Question（自问子问题）
- 需要专业知识时调用 wikipedia_search 工具
- 根据 Observation 生成下一个子问题
- 重复循环，直到所有关键问题都得到解答

以 "Follow-up Question: " 开头，最终用 "Final Answer: " 给出结构化、专业化的分析报告。""",
    debug=True
)
```

教学价值：

这个示例可以看到 Self-Ask 的独特之处——Agent 不是直接思考或先写计划，而是**通过不断自问子问题**来驱动整个流程。每一次 Follow-up Question 都让决策过程更清晰、可追溯。通过同一个旅行规划任务的三种框架对比（ReAct、Plan-and-Execute、Self-Ask），学生能深刻理解不同推理模式如何影响 Agent 的思考路径和最终输出效果，为后续自主决策模式的讲解做好准备。

4.3.6 Self-Ask 与其他代理框架的对比

特性	Self-Ask 代理	ReAct 代理	工具调用代理
问题分解方式	显式子问题分解	隐式推理步骤	自动工具选择
答案整合	显式综合子答案	隐式整合	直接返回
可解释性	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐
事实核查能力	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐
适用问题类型	比较类、事实核查	推理类、决策类	简单查询
输出结构	高度结构化	半结构化	非结构化

5.金融分析

5.1 Plan-and-Execute框架

5.1.1 核心设计理念

Plan-and-Execute（规划-执行框架）是处理**复杂多步骤任务**的高级 Agent 推理模式。它将问题解决过程明确拆分为两个阶段：

- **先整体规划（Plan）**：让 LLM 从全局视角制定完整、结构化的执行计划
- **再分步执行（Execute）**：严格按照计划逐条落实，每一步可调用工具并收集结果

核心思想：通过"先想清楚再动手"的方式，避免传统 Agent 在长任务中"边走边想"导致的路径偏差或遗漏，从而显著提升复杂任务的可靠性和可控性。

在 LangChain 1.0+ 中，Plan-and-Execute **不再依赖单独的 Planner/Executor 类**，而是通过精心设计的 system_prompt 在 create_agent 中实现，让开发者可以用同一套代码灵活切换不同推理模式。

5.1.2 核心组件

在现代实现中，Plan-and-Execute 主要由以下三部分构成：

1.**Planner（规划阶段）**

- 由 system_prompt 驱动 LLM 先生成完整编号的执行计划
- 不直接调用工具，只负责任务分解和全局统筹
- 输出格式通常以 Plan: 开头

2.**Executor（执行阶段）**

- 严格按照 Planner 生成的步骤逐条执行
- 可调用工具、收集 Observation
- 支持 ReAct 风格的单步思考与行动

3.**工具集（Tools）**

- 专注于单一功能的原子工具（如 wikipedia_search、get_stock_history 等）
- 被 Executor 在执行阶段按需调用

5.1.3 为什么需要 Plan-and-Execute?

问题类型	传统 Agent（ReAct 等）	Plan-and-Execute	优势说明
简单查询	✅ 高效	⚠️ 略显过度	简单任务无需提前规划
多步骤复杂任务	⚠️ 易迷失方向	✅ 出色	全局视角避免局部最优
需要策略规划	❌ 困难	✅ 专业	先制定完整计划
动态调整路径	✅ 灵活	⚠️ 需要良好 prompt 引导	初始计划偏差时调整成本较高

问题类型	传统 Agent（ReAct 等）	Plan-and-Execute	优势说明
长期/长链条任务	❌ 易失败	✅ 可靠	步骤清晰、可控

核心价值：当任务步骤较多、依赖关系复杂时，Plan-and-Execute 的“先规划再执行”模式能大幅提升成功率和可解释性。

5.1.4 典型应用场景

研究分析

- 收集多源数据 → 分析趋势 → 生成报告

旅行规划

- 查询交通 → 查找酒店 → 设计行程 → 预算核算

技术诊断

- 收集日志 → 分析根因 → 提出修复方案

金融/股票分析

- 获取基本信息 → 计算技术指标 → 综合投资建议

项目管理

- 任务分解 → 资源分配 → 执行跟踪 → 风险评估

5.2 股票行情分析实例

```python
import baostock as bs
import pandas as pd
import datetime
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import os

# 加载环境变量
load_dotenv()

# ====================== 1. Baostock 全局登录 ======================
print("正在登录 Baostock...")
login_result = bs.login()
if login_result.error_code != '0':
    raise Exception(f"Baostock 登录失败: {login_result.error_msg}")
print("Baostock 登录成功! \n")

```

```python
# ====================== 2. 多个专业股票分析工具（@tool 官方推荐写法）
======================

@tool
def check_ma_crossover(stock_code: str) -> str:
    """检查股票 3日均线与10日均线的金叉/死叉情况"""
    try:
        end_date = datetime.date.today().strftime('%Y-%m-%d')
        start_date = (datetime.date.today() -
datetime.timedelta(days=20)).strftime('%Y-%m-%d')

        rs = bs.query_history_k_data_plus(
            stock_code, "date,close",
            start_date=start_date, end_date=end_date,
            frequency="d", adjustflag="2"
        )

        data_list = []
        while (rs.error_code == '0') and rs.next():
            data_list.append(rs.get_row_data())

        if len(data_list) < 11:
            return "数据不足，无法计算均线"

        df = pd.DataFrame(data_list, columns=rs.fields)
        df['close'] = df['close'].astype(float)
        df['ma3'] = df['close'].rolling(window=3, min_periods=3).mean()
        df['ma10'] = df['close'].rolling(window=10, min_periods=10).mean()
        df = df.dropna().tail(2)

        if len(df) < 2:
            return "数据不足，无法比较"

        prev_ma3, prev_ma10 = df.iloc[-2]['ma3'], df.iloc[-2]['ma10']
        curr_ma3, curr_ma10 = df.iloc[-1]['ma3'], df.iloc[-1]['ma10']

        if prev_ma3 <= prev_ma10 and curr_ma3 > curr_ma10:
            return "金叉（买入信号）"
        elif prev_ma3 >= prev_ma10 and curr_ma3 < curr_ma10:
            return "死叉（卖出信号）"
        else:
            return "无明显交叉"
    except Exception as e:
        return f"工具执行错误: {str(e)}"


@tool
def get_latest_price(stock_code: str) -> str:
    """获取股票最新收盘价、涨跌幅等实时行情"""
    try:
        end_date = datetime.date.today().strftime('%Y-%m-%d')
        rs = bs.query_history_k_data_plus(
            stock_code, "date,close,open,high,low,pctChg",
```

```
            start_date=end_date, end_date=end_date,
            frequency="d", adjustflag="2"
        )
        data = rs.get_row_data()
        if not data:
            return "暂无今日数据"
        return f"最新收盘价: {data[1]} 元 | 涨跌幅: {data[5]}%"
    except Exception as e:
        return f"工具执行错误: {str(e)}"


@tool
def calculate_rsi(stock_code: str, period: int = 14) -> str:
    """计算 RSI 相对强弱指数（超买超卖判断）"""
    try:
        end_date = datetime.date.today().strftime('%Y-%m-%d')
        start_date = (datetime.date.today() - datetime.timedelta(days=period +
20)).strftime('%Y-%m-%d')

        rs = bs.query_history_k_data_plus(stock_code, "date,close", start_date,
end_date, "d", "2")
        data_list = [rs.get_row_data() for _ in range(rs.data.shape[0]) if rs.next()]
        df = pd.DataFrame(data_list, columns=rs.fields)
        df['close'] = df['close'].astype(float)

        delta = df['close'].diff()
        gain = (delta.where(delta > 0, 0)).rolling(window=period).mean()
        loss = (-delta.where(delta < 0, 0)).rolling(window=period).mean()
        rs_value = gain / loss
        rsi = 100 - (100 / (1 + rs_value))

        latest_rsi = rsi.iloc[-1]
        if latest_rsi > 70:
            status = "超买（可能回调）"
        elif latest_rsi < 30:
            status = "超卖（可能反弹）"
        else:
            status = "中性"
        return f"RSI({period}): {latest_rsi:.2f} → {status}"
    except Exception as e:
        return f"工具执行错误: {str(e)}"


@tool
def get_stock_basic_info(stock_code: str) -> str:
    """获取股票基本信息（名称、行业、上市日期等）"""
    try:
        rs = bs.query_stock_basic(code=stock_code)
        data = rs.get_row_data()
        if not data:
            return "未找到股票基本信息"
        return f"股票名称: {data[1]} | 行业: {data[3]} | 上市日期: {data[4]}"
    except Exception as e:
```

```python
        return f"工具执行错误: {str(e)}"


# ====================== 3. 创建现代智能 Agent ======================
def create_stock_agent():
    """创建智能股票技术分析 Agent（使用 create_agent + 丰富 system_prompt）"""

    tools = [check_ma_crossover, get_latest_price, calculate_rsi, get_stock_basic_info]

    llm = ChatOpenAI(
        temperature=0.2,          # 稍微增加创造性，但保持专业
        model="deepseek-chat"
    )

    # 智能 system_prompt（让 Agent 真正自主决策）
    system_prompt = """你是一位专业的 A 股技术分析专家。
    你的目标是：根据用户提供的股票代码，综合使用各种工具，给出清晰、客观、专业的分析报告。

    【决策规则】
    - 先思考用户需求，自主决定需要调用哪些工具
    - 推荐组合使用工具（例如先查基本信息 → 最新价格 → 均线金叉死叉 → RSI）
    - 严格使用 ReAct 模式：Thought → Action → Observation 循环
    - 最终给出结构化报告，包含：基本信息、最新行情、技术信号、投资建议

    最终答案请以 "Final Answer: " 开头，用中文、清晰的格式回复用户。"""

    agent = create_agent(
        model=llm,
        tools=tools,
        system_prompt=system_prompt,
        debug=True    # 打开调试日志，方便教学观察 Agent 思考过程
    )
    return agent


# ====================== 4. 用户交互主程序 ======================
if __name__ == "__main__":
    print("🔥 智能股票技术分析系统（LangChain 1.0+ 现代版）")
    print("=" * 60)
    print("支持工具：MA金叉死叉、最新价格、RSI、股票基本信息")
    print("提示: 输入股票代码如 sh.600519（贵州茅台） 或 sz.000001")

    agent = create_stock_agent()

    while True:
        stock_code = input("\n请输入股票代码（或输入 'exit' 退出）: ").strip()
        if stock_code.lower() == 'exit':
            break
        if not stock_code.startswith(('sh.', 'sz.')) or len(stock_code) < 8:
            print("❌ 股票代码格式错误！正确示例：sh.600519 或 sz.000001")
            continue

```

```
        print(f"\n🔍 正在分析 {stock_code} ...")
        try:
            response = agent.invoke({"input": f"请全面分析股票 {stock_code} 的技术面情况，包括金叉死叉、RSI、最新价格和基本信息"})
            print("\n" + "=" * 60)
            print(f"📊 {stock_code} 分析报告")
            print("=" * 60)
            print(response["output"])
            print("=" * 60)
        except Exception as e:
            print(f"❌ 分析失败: {str(e)}")

    # 退出时登出 Baostock
    bs.logout()
    print("\n系统已安全退出，感谢使用！")
```